Module 4 – Verification IP with UVM Testbench Architecture

Detailed Study Notes

Contents

1 Introduction to UVM

1.1 What is UVM?

- UVM (Universal Verification Methodology) is a standardized methodology for hardware verification, built on SystemVerilog.
- It provides a set of base classes, macros, and guidelines to create modular, reusable, and scalable testbenches.
- UVM is industry-standard for ASIC/FPGA verification (supported by Synopsys, Cadence, Mentor, and others).

1.2 Why UVM?

- **Reusability** – Components (drivers, monitors, agents) can be reused across projects and different levels of hierarchy (block, subsystem, SoC).
- **Randomization and Constraints** – Powerful constrained-random stimulus generation finds corner cases efficiently.
- **Phased Execution** – Well-defined build, connect, run, and report phases ensure predictable testbench behavior.
- **Factory Pattern** – Override any component type without modifying existing code (e.g., replace a driver with a debug version).
- **Transaction-Level Modeling (TLM)** – Communicates using transactions instead of pin wiggling, increasing simulation speed.
- **Functional Coverage** – Built-in support for coverage-driven verification.

1.3 Need for Verification

- Hardware designs (RTL) must be functionally correct before fabrication; fixing bugs after tape-out is extremely expensive (millions of dollars).
- Verification ensures that the design meets its specification under all legal conditions and corner cases.
- Without proper verification, bugs can cause chip respins, project delays, and product failures.

1.4 Types of Verification

1. **Directed Verification** – Test writer explicitly defines each stimulus value and sequence. Best for simple protocols or initial smoke tests.
2. **Random Verification** – Stimulus is randomly generated without constraints. Can explore many states but may produce illegal inputs.
3. **Constrained Random Verification (CRV)** – Random within user-defined constraints (e.g., address in valid range). Most efficient for finding bugs.
4. **Assertion-Based Verification (ABV)** – Monitors design properties (e.g., "request followed by grant within 2 cycles"). Assertions catch protocol violations immediately.
5. **Formal Verification** – Mathematical proof of correctness (not covered in UVM but complementary).

1.5 UVM Testbench Hierarchy (Typical)

```
Test
  Environment
       Agent (Master)
           Sequencer
           Driver
           Monitor
       Agent (Slave)   (passive)
           Monitor
       Scoreboard
       Coverage Collector (Subscriber)
```

2 Writing and Executing Sequences

2.1 Sequence

- A `uvm_sequence` is a container for generating a stream of transaction-level stimuli.
- Sequences can be simple (e.g., 10 write operations) or complex (e.g., interleaved read/write bursts).
- Sequences are executed on a sequencer, which forwards items to the driver.

2.2 Sequence Item

- A `uvm_sequence_item` (or `uvm_object`) represents a single transaction (e.g., one read/write operation).
- Typical fields: address, data, write/read enable, burst length, etc.
- Items can be randomized with constraints to generate legal transactions.

2.3 Example Sequence Item Definition

```
class my_transaction extends uvm_sequence_item;
    rand bit [31:0] addr;
    rand bit [31:0] data;
    rand bit        write;

    constraint addr_range { addr inside {[0:16'hFFFF]}; }
    constraint data_valid { data != 32'h0; }

    `uvm_object_utils_begin(my_transaction)
        `uvm_field_int(addr, UVM_DEFAULT)
        `uvm_field_int(data, UVM_DEFAULT)
        `uvm_field_int(write, UVM_DEFAULT)
    `uvm_object_utils_end

    function new(string name = "my_transaction");
        super.new(name);
    endfunction
endclass
```

2.4 Randomization

- The `randomize()` method randomly sets all `rand` variables respecting constraints.
- Inline constraints can override default constraints: `` `uvm_do_with(req, {addr == 32'h1234;}) ``.
- `` `uvm_do `` macro combines creation, randomization, and sending of a sequence item.

2.5 Constraint Example

```
constraint addr_c { addr < 16; }   // address must be less than 16
```

- Constraints can be soft (overridable) or hard. Soft: `soft addr == 0;`

2.6 Sequence Flow – Detailed

1. User starts a sequence (e.g., `my_sequence.start(sequencer)`).
2. Sequence creates a transaction item (`‘uvm_create` or `new()`).
3. Item is randomized (explicitly or via `‘uvm_do`).
4. Sequence sends item to sequencer (`start_item()` then `finish_item()`).
5. Sequencer forwards item to driver (when driver requests next item).
6. Driver converts item into pin-level signal transitions.
7. Driver may signal completion back via a response (optional).

2.7 ‘uvm_do Macros Family

Macro	Behavior
`‘uvm_do(SEQ_OR_ITEM)`	Create, randomize, and send. If a sequence, start it; if an item, send it.
`‘uvm_do_with(SEQ_OR_ITEM, CONSTRAINTS)`	Same but adds inline constraints.
`‘uvm_create(SEQ_OR_ITEM)`	Create only (no randomization or send).
`‘uvm_rand_send(SEQ_OR_ITEM)`	Randomize and send (item must already exist).
`‘uvm_send(SEQ_OR_ITEM)`	Send without randomization.

2.8 Response Handling

- Driver can return a response item (e.g., read data, status) using `seq_item_port.put(rsp)`.
- Sequence retrieves response with `get_response(rsp)` or `get_response_item()`.
- Responses allow sequences to adapt based on DUT output (e.g., read-after-write checks).

2.9 Virtual Sequences

- A `virtual sequence` is a sequence that does not run on a physical sequencer; instead, it starts other sequences on multiple sequencers.
- It contains handles to all physical sequencers in the testbench (e.g., `axi_seqr`, `gpio_seqr`).
- Example: a virtual sequence for an SoC where a CPU writes to memory via AXI and then reads via I2C.

3 UVM Macros

3.1 Common Macros – Detailed Explanation

@p0.3p0.65@

Macro	Description and Example
`‘uvm_component_utils(type)`	Registers a component class with the factory; enables factory overrides and config DB.

`‘uvm_component_utils(my_driver)`

‘uvm_object_utils(type) Registers a data object (transaction, sequence) with the factory.
‘uvm_object_utils(my_transaction)
‘uvm_info(msg_id, msg, verbosity) Prints a message with time and hierarchy; verbosity controls visibility (e.g., UVM_MEDIUM).
‘uvm_info("DRV", "Data sent", UVM_LOW)
‘uvm_error(msg_id, msg) Prints an error; increments error count. Test can pass/fail based on error threshold.
‘uvm_fatal(msg_id, msg) Prints a fatal message and calls $finish. Stops simulation.
‘uvm_warning(msg_id, msg) $Prints a warning (non-fatal)$.
‘uvm_field_int(field, flags) $Automates copying, printing, comparing, packing of integer fields in$d

3.2 Factory Override

- The factory is a central registry that creates objects/component classes.
- Override: replace a base type with a derived type for all or specific instances.
- Useful for substituting a debug driver, error injector, or coverage monitor without changing testbench structure.

3.3 Example of Type Override

```
class my_driver extends uvm_driver #(my_transaction);
    ‘uvm_component_utils(my_driver)
    // ...
endclass

// In test's build_phase:
set_type_override_by_type(uvm_driver::get_type(), my_driver::
   get_type());
```

3.4 Instance Override

```
set_inst_override_by_type("env.agent.driver", uvm_driver::get_type
   (),
                          my_driver::get_type());
```

3.5 Configuration Database (‘uvm_config_db)

- uvm_config_db is a global, type-safe, hierarchical configuration store for sharing settings between components.
- Typical use: setting virtual interfaces, enabling debug modes, passing test parameters.
- Methods: set() and get().

```
// In top testbench:
uvm_config_db#(virtual my_if)::set(null, "uvm_test_top.env.agent",
    "vif", my_if);
```

```
3
4 // In agent:
5 uvm_config_db#(virtual my_if)::get(this, "", "vif", vif);
```

3.6 Command-line Processor

- `uvm_cmdline_processor` parses arguments passed to simulator (e.g., `+UVM_TESTNAME=my_test`).
- Provides convenience methods like `get_arg_value()`.

4 Virtual Sequences and Arbitration

4.1 Virtual Sequence – In-depth

- Virtual sequences are `uvm_sequence` objects that do not have an associated sequencer (parameter to `‘uvm_object_utils`, not `‘uvm_component_utils`).
- They contain `uvm_sequencer` handles (often set via config DB or explicit assignment).
- Inside the virtual sequence's `body()`, you start child sequences on specific sequencers.

4.2 Example Virtual Sequence Body

```
1  class vseq_base extends uvm_sequence;
2      ‘uvm_object_utils(vseq_base)
3      axi_sequencer   m_axi_seqr;
4      gpio_sequencer  m_gpio_seqr;
5
6      task body();
7          axi_write_seq wseq;
8          axi_read_seq  rseq;
9          gpio_cfg_seq  gseq;
10         ‘uvm_do_on(wseq, m_axi_seqr)
11         ‘uvm_do_on(gseq, m_gpio_seqr)
12         ‘uvm_do_on(rseq, m_axi_seqr)
13     endtask
14 endclass
```

4.3 Arbitration Methods on a Sequencer

When multiple sequences request access to the same physical sequencer, arbitration decides who gets the next item.

Method	Behavior
UVM_SEQ_ARB_FIFO	First-in, first-out order.
UVM_SEQ_ARB_WEIGHTED	Each sequence has a weight; higher weight gets more accesses.
UVM_SEQ_ARB_RANDOM	Randomly selects among ready sequences.
UVM_SEQ_ARB_STRICT_FIFO	Only one sequence runs until empty, then next.
UVM_SEQ_ARB_STRICT_RANDOM	Randomly pick a sequence and run it to completion.

4.4 Setting Arbitration on a Sequencer

```
sequencer.set_arbitration(UVM_SEQ_ARB_WEIGHTED);
```

4.5 Locking and Grab

- `lock()` – sequence requests exclusive access to sequencer; waits until previous lock is released.
- `grab()` – immediately acquires exclusive access, even if another sequence holds lock (dangerous).
- `unlock()` / `ungrab()` – release exclusivity.
- Useful for atomic operations that cannot be interleaved (e.g., flash programming sequence).

4.6 Deadlock – Causes and Prevention

- **Cause** – Sequence A waits for response from Sequence B, and B waits for A's resource. Or both lock the same sequencer.
- **Prevention** – Avoid circular dependencies, use timeouts (`‘uvm_do_with(..., {timeout == 1000ns;})`), and model clear sequence flow.
- **UVM detection** – A hang will cause simulation to run forever; use watchdog threads.

5 Layered UVM Testbench

5.1 UVM Component Overview

5.1.1 Test (uvm_test)

- Top-level component; configures testbench parameters via config DB and builds the environment.
- Chooses which virtual sequence to run in the run_phase.
- Typically the only component that is explicitly instantiated outside the factory.

5.1.2 Environment (uvm_env)

- Container for all active verification components (agents, scoreboard, coverage, etc.).
- Builds and connects them in the build and connect phases.
- May contain multiple agents (master, slave, monitor-only) for multi-interface verification.

5.1.3 Agent (`uvm_agent`)

- Groups together the driver, sequencer, and monitor for a specific interface.
- Has an `is_active` parameter (UVM_ACTIVE vs UVM_PASSIVE). Active = driver+sequencer+monitor; Passive = monitor only.
- Monitors can be passive agents without drivers (e.g., bus analyzer).

5.1.4 Sequencer (`uvm_sequencer`)

- Broker between sequences and driver. Receives items from sequences and hands them to the driver on request.
- Implements arbitration, locking, and response forwarding.
- Parameterized with the transaction type.

5.1.5 Driver (`uvm_driver`)

- Listens to sequencer for next transaction, converts it to pin-level signals using a virtual interface.
- Handles clock edge synchronization, protocol timings, reset, and optionally sends responses.
- Typically extends `uvm_driver#(REQ, RSP)`.

5.1.6 Monitor (`uvm_monitor`)

- Observes DUT output signals via virtual interface; waits for transaction-level events (e.g., bus cycle completed).
- Assembles observed signal values into a transaction object.
- Broadcasts the transaction to analysis subscribers (scoreboard, coverage collector) using an analysis port.

5.1.7 Scoreboard (`uvm_scoreboard`)

- Receives expected transactions (from predictor or reference model) and actual transactions (from monitor).
- Compares them for equality, may account for out-of-order completion.
- Reports mismatches as errors.

5.1.8 Subscriber (`uvm_subscriber`)

- Implements the analysis interface (e.g., `write()` method).
- Collects transactions for functional coverage, performance metrics, or logging.
- Typically extends `uvm_subscriber#(T)`.

5.2 TLM Ports and Connections

Port/Export	Purpose
`uvm_analysis_port`	Broadcast to multiple subscribers (many-to-many).
`uvm_analysis_export`	Accepts connections from analysis ports.
`uvm_analysis_imp`	Implementation of analysis interface (in subscriber).
`uvm_tlm_analysis_fifo`	FIFO buffer that stores transactions; connects analysis port to analysis imp.
`uvm_blocking_get_port`	Request data from a provider (blocking).
`uvm_nonblocking_put_port`	Push data without blocking (try_put).

5.3 Data Flow Example

```
// Monitor writes to analysis port
class my_monitor extends uvm_monitor;
    uvm_analysis_port #(my_transaction) ap;
    // ...
    task run_phase();
        forever begin
            my_transaction tx = my_transaction::type_id::create("
               tx");
            // collect signals from interface
            ap.write(tx);
        end
    endtask
endclass

// Scoreboard implements the analysis imp
class my_scoreboard extends uvm_scoreboard;
    uvm_analysis_imp #(my_transaction, my_scoreboard) analysis_imp
       ;
    function void write(my_transaction tx);
        // compare with expected
    endfunction
endclass

// In environment's connect_phase:
monitor.ap.connect(scoreboard.analysis_imp);
```

6 UVM Phases

6.1 Overview of Phases

UVM phases are hierarchical methods called automatically on all components. They ensure predictable setup, execution, and tear-down.

6.2 Build Phase (`build_phase`)

- Called top-down (test → env → agent → driver/monitor).

- Construct sub-components using `type_id::create()`.
- Get configuration from config DB (e.g., virtual interface).
- **Important:** Do not communicate with DUT here; DUT may not be ready.

6.3 Connect Phase (`connect_phase`)

- Called after build_phase, bottom-up (drivers first, then up to test).
- Establish TLM connections (analysis ports to exports, FIFO connections).
- Connect virtual interfaces if not done in build.

6.4 Run Phase (`run_phase`)

- Main simulation phase; runs in parallel for all components.
- Typically raise objections, start sequences, wait for completion, then drop objections.
- Use `phase.raise_objection(this)` and `drop_objection(this)` to control termination.

6.5 Run-time Phases (12 sub-phases)

UVM provides `pre_reset`, `reset`, `post_reset`, `pre_configure`, `configure`, `post_configure`, `pre_main`, `main`, `post_main`, `pre_shutdown`, `shutdown`, `post_shutdown`.
These are optional but useful for protocol-driven tests (apply reset, configure registers, run main test, shut down).

6.6 Extract Phase (`extract_phase`)

- Called after run phase completes (and all objections dropped).
- Collect final statistics, dump coverage data, gather scoreboard counts.

6.7 Check Phase (`check_phase`)

- Perform end-of-test checks (e.g., all expected data received, no pending transactions).
- Assert certain conditions; report failures if not met.

6.8 Report Phase (`report_phase`)

- Generate final test report (pass/fail, error counts, coverage numbers).
- Can be customized to produce JUnit XML or custom logs.

6.9 Final Phase (`final_phase`)

- Perform any last cleanup (e.g., file close, simulation statistics).
- Rarely used.

6.10 Objection Mechanism

- UVM simulation ends when all objections are dropped.
- Components raise objections at start of run_phase and drop at end.
- `phase.raise_objection(this, "Main sequence started");`
- `phase.drop_objection(this, "Main sequence done");`
- If no objections raised, simulation stops immediately.

6.11 Phase Diagram

```
build --> connect --> (optional reset/configure/main phases) -->
     extract --> check --> report --> final
```

All run-time phases (main, etc.) are children of the run phase and can be overridden.

7 Driver and Sequencer

7.1 Driver Implementation Details

- Extends `uvm_driver#(REQ, RSP)`.
- Override `run_phase()` method.
- Task `get_next_item(req)` from sequencer (blocking).
- Drive transaction onto interface (using virtual interface clocking).
- Call `item_done()` when driving complete (optionally with a response).

7.2 Example Driver Skeleton

```
class my_driver extends uvm_driver#(my_transaction);
    `uvm_component_utils(my_driver)
    virtual my_if vif;

    function void build_phase(uvm_phase phase);
        if(!uvm_config_db#(virtual my_if)::get(this, "", "vif",
           vif))
            `uvm_fatal("DRV", "Virtual interface not set")
    endfunction

    task run_phase(uvm_phase phase);
        my_transaction req;
        forever begin
            seq_item_port.get_next_item(req);
            drive_transaction(req);
            seq_item_port.item_done();
        end
    endtask

    task drive_transaction(my_transaction t);
        @(posedge vif.clk);
        vif.addr <= t.addr;
        vif.data <= t.data;
```

```
        @(posedge vif.clk);
        vif.en <= 1;
        // ...
    endtask
endclass
```

7.3 Virtual Interface – Detailed

- A SystemVerilog interface that groups DUT signals (clock, data, control).
- Virtual interface is a pointer to an actual interface instance (set from top testbench).
- Allows UVM components to drive/monitor signals without *roothierarchyreferences*.

- Must be used with care for clocking blocks and modports.

7.4 Sequencer Usage

- Typically parameterized: `uvm_sequencer#(my_transaction)`.
- Does not need custom implementation except for advanced arbitration.
- Can be extended to add custom policy (e.g., response queues).

8 Monitor, Agent, Scoreboard

8.1 Monitor Details

- Passive component: observes DUT output without driving.
- Contains a virtual interface; samples signals on clock edges.
- Detects transaction start/end conditions (e.g., valid and ready asserted).
- Creates a transaction object, fills fields, then broadcasts via analysis port.

8.2 Monitor Example (AXI-like)

```
class my_monitor extends uvm_monitor;
    `uvm_component_utils(my_monitor)
    uvm_analysis_port #(my_transaction) ap;
    virtual my_if vif;

    task run_phase(uvm_phase phase);
        forever begin
            @(posedge vif.clk);
            if (vif.valid && vif.ready) begin
                my_transaction tx = my_transaction::type_id::
                    create("tx");
                tx.addr = vif.addr;
                tx.data = vif.data;
                tx.write = vif.write;
                ap.write(tx);
            end
        end
```

```
    endtask
endclass
```

8.3 Agent Implementation

```
class my_agent extends uvm_agent;
    `uvm_component_utils(my_agent)
    my_driver    driver;
    my_sequencer sequencer;
    my_monitor   monitor;

    function void build_phase(uvm_phase phase);
        if (get_is_active() == UVM_ACTIVE) begin
            driver = my_driver::type_id::create("driver", this);
            sequencer = my_sequencer::type_id::create("sequencer",
                this);
        end
        monitor = my_monitor::type_id::create("monitor", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        if (get_is_active() == UVM_ACTIVE)
            driver.seq_item_port.connect(sequencer.seq_item_export
               );
    endfunction
endclass
```

8.4 Scoreboard – Deep Dive

- Typical scoreboard uses queues or associative arrays to store expected transactions.
- May reorder transactions if DUT can complete out-of-order (e.g., multiple channels).
- Common pattern: predictor (reference model) generates expected transactions and sends them to scoreboard.
- Real monitor sends actual transactions; scoreboard matches them using a tag (e.g., transaction ID).

8.5 Scoreboard Example with ID Matching

```
class my_scoreboard extends uvm_scoreboard;
    uvm_analysis_imp #(my_transaction, my_scoreboard) exp_imp;
    uvm_analysis_imp #(my_transaction, my_scoreboard) act_imp;
    local my_transaction exp_q[$];

    function void write_exp(my_transaction t);
        exp_q.push_back(t);
    endfunction

```

```
    function void write_act(my_transaction t);
        my_transaction expected;
        // find matching ID or FIFO order
        expected = exp_q.pop_front();  // simplified
        if (!t.compare(expected))
            `uvm_error("SCB", $sformatf("Mismatch: got %s, exp %s",
                t.convert2string(), expected.convert2string()))
    endfunction
endclass
```

8.6 Functional Coverage

: Measure verification completeness – which legal scenarios have been exercised.

- Defined using covergroups and coverpoints in a coverage collector component (often a subscriber).
- Sample data from transactions (e.g., address ranges, opcodes, data patterns).
- Use `cross` to cover combinations (e.g., address = 0xFF and write = 1).
- Report coverage at end of test; typical goal is 100% for critical cross products.

8.7 Coverage Types

Type	Description
Statement	Every line of RTL executed at least once (code coverage).
Branch	Every branch condition taken and not taken (code coverage).
Toggle	Every bit in design toggled 0→1 and 1→0 (code coverage).
Functional	User-defined interesting scenarios (specification-driven).

8.8 Coverage Example in UVM

```
class coverage_collector extends uvm_subscriber #(my_transaction);
    `uvm_component_utils(coverage_collector)
    covergroup cg;
        addr_cp: coverpoint tx.addr { bins low = {[0:255]}; bins
            high = {[256:1023]}; }
        write_cp: coverpoint tx.write;
        cross addr_cp, write_cp;
    endgroup

    function write(my_transaction t);
        this.tx = t;
        cg.sample();
    endfunction
endclass
```

9 Subscriber and Virtual Sequencer

9.1 Subscriber – Role and Implementation

- A subscriber is a component that implements `write(T t)` method.
- Typically extends `uvm_subscriber#(T)` or `uvm_scoreboard`.
- Used for coverage collection, transaction logging, performance monitoring.
- Connected to analysis ports of monitors or predictors.

9.2 Virtual Sequencer – More Details

- A component (extends `uvm_sequencer`) that contains handles to physical sequencers.
- Does not process sequences directly, but routes virtual sequences to appropriate physical sequencers.
- Required for virtual sequences because sequences need a sequencer to run on, even if they only start other sequences.
- Typical virtual sequencer instantiated in environment or test, then config DB used to assign physical sequencer handles.

9.3 Virtual Sequencer Example

```
class virtual_sequencer extends uvm_sequencer;
    `uvm_component_utils(virtual_sequencer)
    axi_sequencer    m_axi_seqr;
    gpio_sequencer   m_gpio_seqr;

    function void build_phase(uvm_phase phase);
        // handles will be set via config DB from test / env
        uvm_config_db#(axi_sequencer)::get(this, "", "axi_seqr",
           m_axi_seqr);
        uvm_config_db#(gpio_sequencer)::get(this, "", "gpio_seqr",
            m_gpio_seqr);
    endfunction
endclass
```

9.4 Connecting Virtual Sequence to Virtual Sequencer

```
// In test's build_phase:
virtual_sequencer vseqr = virtual_sequencer::type_id::create("
   vseqr", this);
vseqr.m_axi_seqr = env.axi_agent.sequencer;
vseqr.m_gpio_seqr = env.gpio_agent.sequencer;

// In run_phase:
my_virtual_sequence vseq = my_virtual_sequence::type_id::create("
   vseq");
vseq.start(vseqr);
```

9.5 Applications of Virtual Sequences

- **SoC Verification** – Coordinate CPU, DMA, interrupt controllers, peripherals.
- **Multi-Interface Verification** – e.g., AXI master, AXI slave, APB configuration concurrently.
- **Bus Protocol Verification** – Master and slave agents on the same bus with backpressure.
- **Scenario Generation** – Power-up sequence, error injection, reset handling across multiple interfaces.

10 Advanced UVM Topics (Brief Overview)

10.1 TLM 2.0 Sockets

- TLM 2.0 provides blocking and non-blocking transport interfaces.
- Used primarily for loosely-timed or approximately-timed models (virtual prototypes).
- UVM provides `uvm_tlm_transport_socket`, `uvm_tlm_analysis_socket`.

10.2 UVM Register Layer (RAL)

- Standardized model for describing and verifying registers and memories.
- Provides frontdoor (bus) and backdoor (direct) access to DUT registers.
- Automates read/write sequences, mirroring, and field-level checking.

10.3 Sequence Library

- `uvm_sequence_library` collects multiple sequences and selects one randomly or via configuration.
- Simplifies running many different sequences without modifying test code.

10.4 Reporting and Verbosity Levels

- Levels: `UVM_NONE`, `UVM_LOW`, `UVM_MEDIUM`, `UVM_HIGH`, `UVM_FULL`, `UVM_DEBUG`.
- Set globally via `+UVM_VERBOSITY=UVM_MEDIUM` or per-component.

10.5 Coding Guidelines for UVM Testbenches

- Use factory macros for all components and objects.
- Separate stimulus generation (sequences) from protocol implementation (driver).
- Avoid absolute hierarchical paths; use config DB and get_full_name().
- Keep transactions DUT-agnostic (signals abstracted).
- Use virtual sequences for high-level scenarios.
- Raise/drop objections properly to avoid premature test termination.

11 Complete UVM Testbench Example Skeleton

```
// 1. Interface
interface my_if (input clk, resetn);
    logic [31:0] addr, data;
    logic valid, ready, write;
    clocking cb @(posedge clk);
        output addr, data, valid, write;
        input ready;
    endclocking
    modport DRIVER (clocking cb);
    modport MONITOR (input addr, data, valid, ready, write);
endinterface

// 2. Transaction
class my_transaction extends uvm_sequence_item;
    rand bit [31:0] addr, data;
    rand bit write;
    `uvm_object_utils_begin(my_transaction)
        `uvm_field_int(addr, UVM_ALL_ON)
        `uvm_field_int(data, UVM_ALL_ON)
        `uvm_field_int(write, UVM_ALL_ON)
    `uvm_object_utils_end
endclass

// 3. Driver (similar to previous example)

// 4. Monitor (similar to previous example)

// 5. Agent (active)

// 6. Environment
class my_env extends uvm_env;
    my_agent m_agent;
    my_scoreboard m_sb;
    coverage_collector m_cov;
    `uvm_component_utils(my_env)
    function void build_phase(uvm_phase phase);
        m_agent = my_agent::type_id::create("m_agent", this);
        m_sb    = my_scoreboard::type_id::create("m_sb", this);
        m_cov   = coverage_collector::type_id::create("m_cov",
           this);
    endfunction
    function void connect_phase(uvm_phase phase);
        m_agent.monitor.ap.connect(m_sb.act_imp);
        m_agent.monitor.ap.connect(m_cov.analysis_imp);
        // connect predictor if any
    endfunction
endclass

// 7. Test
class my_test extends uvm_test;
```

```
50      my_env env;
51      `uvm_component_utils(my_test)
52      function void build_phase(uvm_phase phase);
53          env = my_env::type_id::create("env", this);
54          uvm_config_db#(virtual my_if)::set(this, "env.*", "vif",
                top.my_if);
55      endfunction
56      task run_phase(uvm_phase phase);
57          my_virtual_sequence vseq;
58          phase.raise_objection(this);
59          vseq = my_virtual_sequence::type_id::create("vseq");
60          vseq.start(env.m_agent.sequencer);
61          phase.drop_objection(this);
62      endtask
63  endclass
```

12 Common Debugging Tips

- Use `` `uvm_info `` with different verbosity to trace execution without cluttering logs.
- Enable `+UVM_PHASE_TRACE` to see phase progression.
- Use `+UVM_OBJECTION_TRACE` to track objection raises/drops.
- `uvm_top.print_topology()` prints testbench hierarchy.
- `transaction.print()` prints field values.
- Simulator GUI with UVM debug plugin helps inspect sequences and TLM connections.

13 Important Formulas (for performance metrics)

13.1 Verification Efficiency

$$\text{Bug Detection Rate} = \frac{\text{Number of unique bugs found}}{\text{Total simulation cycles}} \tag{1}$$

13.2 Coverage Closure

$$\text{Coverage Progress} = \frac{\text{Current covered points}}{\text{Total coverpoints}} \times 100\% \tag{2}$$

13.3 Simulation Performance

$$\text{Simulation Speed} = \frac{\text{Total cycles simulated}}{\text{Simulation wall-clock time}} \tag{3}$$